

Aula 04

Fundamentos do React

Objetivos

Entender os conceitos fundamentais do React: componentes, props, state e hooks

Configurar o front-end (Axios + CORS) para consumir a API REST criada na Aula 03

Implementar uma tela de listagem de Usuario consumindo o back-end Spring Boot via H2

Agenda

1. Revisão
2. Conceitos
3. Demonstração
4. Prática
5. Exercícios

Revisão da Aula Anterior

Transformamos Usuario em uma **entidade JPA** (`@Entity` , `@Id` , `@GeneratedValue`)

Criamos o `UsuarioRepository` extends `JpaRepository<Usuario, Long>` Substituímos o `ArrayList` em memória por persistência real no **H2** Endpoints `GET /api/usuarios` e `POST /api/usuarios` já funcionam com dados reais salvos em banco

Hoje: sair do Postman/Insomnia e consumir esses endpoints direto do **React**.

Conceitos Fundamentais

O que é o React?

Biblioteca JavaScript/TypeScript para construir **interfaces de usuário**

Baseada em **componentes**: pedaços reutilizáveis de UI

Utiliza **JSX**: sintaxe que mistura HTML dentro do JavaScript/TypeScript

Atualiza a tela automaticamente quando os **dados mudam** (renderização reativa)

Componentes

Um componente React é, na prática, uma **função** que retorna JSX

Pode ser reaproveitado várias vezes com dados diferentes

```
function BemVindo() {  
  return <h1>Olá, Programação Web II!</h1>;  
}
```

Componentes podem ser **compostos**: um componente pode renderizar outros componentes dentro dele

Props (propriedades)

Forma de **passar dados de um componente pai para um componente filho**

São **somente leitura** (o filho não deve alterar as props recebidas)

```
function UsuarioItem({ nome, email }: { nome: string; email: string }) { return <li>{nome} —  
  {email}</li>;  
}
```

// Uso:

```
<UsuarioItem nome="João" email="joao@example.com" />
```

State (estado)

Dados **internos** do componente que podem mudar ao longo do tempo

Quando o state muda, o React **re-renderiza** o componente automaticamente

Gerenciado através do hook useState

```
import { useState } from "react";

function Contador() {
  const [contagem, setContagem] = useState(0);

  return (
    <button onClick={() => setContagem(contagem + 1)}>
```

```
    Cliquei {contagem} vezes  
  </button>  
);  
}
```

Spring Boot + React 7

Programação Web II

Hooks

Funções especiais que permitem "conectar" **estado** e **ciclo de vida** a componentes funcionais

Sempre começam com **use**

Hook	Para que serve
useState	Guardar e atualizar dados que mudam

useEffect	Executar código em resposta a eventos do ciclo de vida (ex: buscar dados ao carregar a tela)
-----------	--

Spring Boot + React 8

Programação Web II

useEffect

```
import { useEffect, useState } from "react";

function Relogio() {
  const [hora, setHora] = useState(new Date());

  useEffect(() => {
    const id = setInterval(() => setHora(new Date()), 1000);
    return () => clearInterval(id); // limpeza ao desmontar o componente }, []); // [] = executa
    apenas uma vez, ao montar o componente

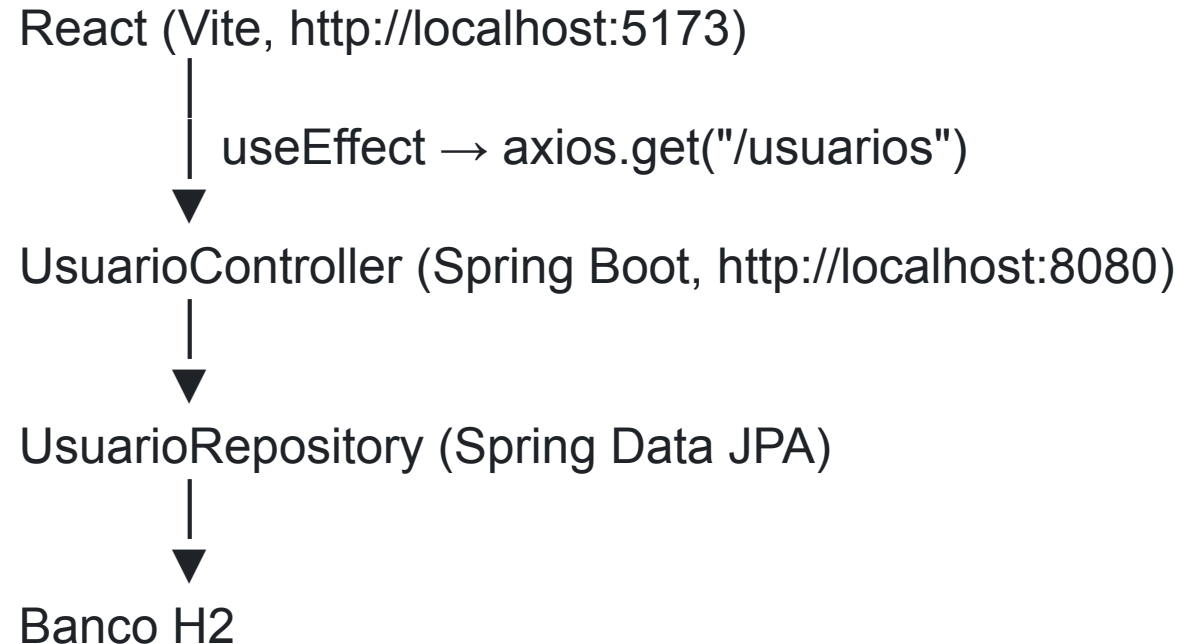
  return <p>{hora.toLocaleTimeString()}</p>;
}
```

É dentro do `useEffect` com array `[]` que vamos **buscar os dados da API** assim que a tela carregar.

Spring Boot + React 9

Programação Web II

Arquitetura / Fluxo



Resposta JSON —► setUsuarios(dados) —► re-render da tela

Front-end e back-end rodam em **portas diferentes** ⇒ é preciso configurar **CORS** no Spring Boot.

Spring Boot + React 10

Programação Web II

Demonstração

1. Instalar o Axios no front-end

```
cd frontend  
npm install axios
```

2. Configurar CORS no Spring Boot

@RestController

```
@RequestMapping("/api")
@CrossOrigin(origins = "http://localhost:5173")
public class UsuarioController {
    // ...
}
```

Spring Boot + React 11

Programação Web II

Demonstração — cliente HTTP (api.ts)

```
// src/services/api.ts
import axios from "axios";

const api = axios.create({
  baseURL: "http://localhost:8080/api",
});

export default api;
```

Centralizar a configuração do Axios evita repetir a URL base em todo

componente. Spring Boot + React 12

Programação Web II

Demonstração — tipo **Usuario**

```
// src/types/Usuario.ts
export interface Usuario {
  id: number;
  nome: string;
  username: string;
```

```
    email: string;  
}
```

Não incluímos o campo senha : lembre que ele está com `@JsonIgnore` no back-end e nunca vem na resposta da API.

Spring Boot + React 13

Programação Web II

Demonstração - `Usuarioltem.tsx` (props)

```
// src/components/Usuarioltem.tsx  
import type { Usuario } from "../types/Usuario";  
  
interface UsuarioltemProps {  
    usuario: Usuario;  
}
```

```
function Usuarioltem({ usuario }: UsuarioltemProps) {  
  return (  
    <li>  
      <strong>{usuario.nome}</strong> ({usuario.username}) — {usuario.email} </li>  
    );  
  }  
  
export default Usuarioltem;
```

Spring Boot + React 14

Programação Web II

Demonstração — UsuarioList.tsx

```
// src/components/UsuarioList.tsx  
import { useEffect, useState } from "react";  
import api from "../services/api";  
import type { Usuario } from "../types/Usuario";  
import Usuarioltem from "../Usuarioltem";  
  
function UsuarioList() {  
  const [usuarios, setUsuarios] = useState<Usuario[]>([]);
```

```

useEffect(() => {
  api.get<Usuario[]>("/usuarios").then((resposta) => {
    setUsuarios(resposta.data);
  });
}, []);

return (
  <ul>
    {usuarios.map((usuario) => (
      <UsuarioItem key={usuario.id} usuario={usuario} />
    ))}
  </ul>
);
}

```

Spring Boot + React 15
Programação Web II

Demonstração — App.tsx

```
import UsuarioList from "../components/UsuarioList";
```

```
function App() {
  return (
```

```
    <div>
      <h1>Usuários cadastrados</h1>
      <UsuarioList />
    </div>
  );
}

export default App;
```

Spring Boot + React 16

Programação Web II

Atividade Prática

Objetivo

Criar, no projeto React, uma tela que lista os usuários cadastrados, consumindo a API do back-end feita na Aula 03.

Passos

1. Instalar o Axios no front-end (`npm install axios`)
2. Configurar `@CrossOrigin` no `UsuarioController` (ou uma `CorsConfiguration` global)
3. Criar `src/services/api.ts` com a instância do Axios (`baseUrl`)
4. Criar `src/types/Usuario.ts` com a interface do usuário
5. Criar o componente `UsuarioList.tsx` usando `useState` + `useEffect` para `Spring Boot + React`

17r ri

Programação Web II

Exercícios

1. Criar `PermissaoList.tsx` , consumindo `GET /permissoes` (entidade criada na Aula 02/03)

2. Criar a listagem em React da entidade de seu interesse escolhida anteriormente (Produto , Pedido , Cliente , etc.)
3. Adicionar um estado de **carregando** (loading) exibido enquanto a requisição não retorna
4. Tratar o caso de **erro** (ex: back-end desligado) exibindo uma mensagem para o usuário
5. Ao final, comitar e subir as alterações no repositório do GitHub

Spring Boot + React 18

Programação Web II

Referências

Spring Boot Documentation

React Documentation — <https://react.dev/learn>

Axios Documentation — <https://axios-http.com/docs/intro>

Material complementar da disciplina

Conteúdo: CRUD Completo — formulários controlados em React (criar/editar) e os métodos POST , PUT e DELETE no Spring Boot

Preparação: deixar a listagem de Usuario desta aula funcionando, com back end e front-end rodando simultaneamente